

MIDDLEWARE VS FILTERS

LET'S UNDERSTAND



Written by
Petros Sargsian



WHAT IS MIDDLEWARE?

Middleware processes HTTP requests in a pipeline. Each component can process the request before passing it to the next component.

- Executes on every request
- Can short-circuit the pipeline
- Order matters • Configured in Program.cs

```
public class RequestLoggingMiddleware
{
    public async Task InvokeAsync()
    {
        Log($"Request: {path}");
        await _next(context);
    }
}
```



Written by
Petros Sargsian



MIDDLEWARE PIPELINE

```
var app = builder.Build();

// 1. Exception Handling (should be first)
app.UseExceptionHandler("/Error");
app.UseHsts();

// 2. HTTPS Redirection
app.UseHttpsRedirection();

// 3. Static Files
app.UseStaticFiles();

// 4. Routing
app.UseRouting();

// 5. CORS (if needed)
app.UseCors();

// 6. Authentication
app.UseAuthentication();

// 7. Authorization
app.UseAuthorization();

// 8. Session (if needed)
app.UseSession();

// 9. Response Caching (if needed)
app.UseResponseCaching();

// 10. Endpoints (should be last)
app.MapControllers();
app.MapRazorPages();

app.Run();
```



Written by
Petros Sargsian



WHAT ARE FILTERS?

Filters run code before or after specific stages in the MVC action pipeline. They only execute for MVC/Web API requests.

- Execute only for MVC actions
- Applied via attributes
- Multiple types available • Action-specific control

```
public class ValidateModelFilter : IActionFilter
{
    public void OnActionExecuting(context)
    {
        if (!context.ModelState.IsValid)
            context.Result = new BadRequestResult();
    }
}
```



Written by
Petros Sargsian



TYPES OF FILTERS

Five main filter types in ASP.NET Core:

- Authorization: Runs first, checks permissions
- Resource & Action: Before/after execution
- Exception & Result: Handles errors and results

```
[Authorize]
[ServiceFilter(typeof(LogFilter))]
public class ProductsController
{
    [ValidateModel]
    public IActionResult Create(Product p)
    { return Ok(p); }
}
```



Written by
Petros Sargsian



WHEN TO USE MIDDLEWARE

- Authentication & Authorization
- Global Error Handling
- Logging
- Monitoring
- CORS & Static Files

```
var app = builder.Build();  
app.UseHttpsRedirection();  
app.UseAuthentication();  
app.UseAuthorization();  
app.Use(async (ctx, next) =>  
{ await next.Invoke(); });  
app.Run();
```



Written by
Petros Sargsian



WHEN TO USE FILTERS

- Model Validation
- Action-Specific Caching
- Performance Tracking
- Custom Authorization

```
[Route("api/orders")]
public class OrdersController
{
    [ValidateModel]
    public IActionResult Create() { }

    [ResponseCache(Duration = 60)]
    public IActionResult Get() { }
}
```



Written by
Petros Sargsian



KEY DIFFERENCES

- **Scope:**
 - Middleware runs on all requests
 - Filters only on MVC/API
- **Configuration:**
 - Middleware in Program.cs
 - Filters via attributes
- **Use When:**
 - Middleware for app-wide
 - Filters for action-level



Written by
Petros Sargsian





VOILA!

Follow for more content like this.

↔ Repost

+ Follow me